PacketHunter: Multi-threaded Deep Packet Inspection Engine

Sarvesh Shinde
Department of Information Technology
Email: sarveshshinde360@email.com

***Abstract*—The paper describes the design and realization of a *PacketHunter*, a multi-threaded DPI (Deep Packet Inspection) engine to provide effective traffic analysis on the network. In particular, this system operates with packets from different protocols like TCP, UDP, and identifies application data (for example, TLS Server Name Indication (SNI)). It makes use of FiveTuple tracking to perform connection-level traffic analysis. Also, a rule-based engine to classify traffic in accordance with certain rules is considered. The multi-threaded structure of the proposed architecture was chosen according to the producer/consumer approach. As shown by experiments, this architecture allows us to achieve a 2–3x increase in throughput under heavy load.**

***Index Terms*—Deep Packet Inspection (DPI), Network Traffic Analysis, Flow Tracking, Multithreading, TLS SNI, Packet Classification**

I. Introduction

However, with the fast growth in network-based applications, the traffic becomes more complex and bulky. Modern communication infrastructures use a set of different protocols such as TCP, UDP, HTTP, TLS etc. Hence, the task of traffic classification becomes difficult since conventional approaches based on packet filtering techniques fail in detecting application-level protocols and performing analysis of network streams.

An advanced approach for packet inspection is Deep Packet Inspection (DPI). This method allows extracting and analyzing not only the packet headers but also its payload. Therefore, DPI allows gaining additional visibility into the network activity. DPI engines are used in many software applications such as network monitoring systems, intrusion detection systems, traffic managers and others.

This work describes a multi-threaded implementation of the Deep Packet Inspection engine named *PacketHunter*. This engine inspects network packets and extracts application-level information such as TLS Server Name Indication (SNI). Moreover, this information allows classifying traffic based on certain predefined rules. Flow-based tracking using FiveTuple (five-tuple includes source/destination IP addresses, port numbers and the used protocol) maintains connection-level states of analyzed traffic.

Moreover, the developed system uses a multi-threaded architecture inspired by the producer-consumer model. This feature allows increasing scalability of packet inspection under the load.

II. Problem Statement

Today's modern computer networks see a lot of traffic that comes in many different flavors: TCP, UDP, HTTP, and TLS. The traditional packet inspection technique is based on inspecting only packets' headers. This results in inability to analyze applications and detect anomalies on network flows.

However, due to the rise of network traffic and increased security demands, it becomes necessary to introduce technologies that would be able to scan the contents of the packets in order to obtain useful information such as application identification. High speed networks, however, call for efficient and fast processing techniques that would allow for analyzing the traffic with no loss of performance.

Thus, the problem of the present paper is designing an efficient and scalable DPI (Deep Packet Inspection) engine capable of inspecting packet contents and identifying useful information (e.g., TLS SNI) in the course of flow-based connection tracking.

III. Literature Survey

Deep Packet Inspection has received significant attention in scientific literature as well as industry practice over the years. Initial attempts to analyze network traffic involved the development of mechanisms for monitoring and detecting anomalies in the process of communication [1], where the author emphasizes the necessity to move beyond simple packet header filtering to perform deep inspections.

The advancement of network infrastructure technologies prompted DPI to become an integral part of next-generation firewalls and intrusion detection systems. In his paper [2], Varghese focuses on efficient algorithms for network packets processing at high speeds.

In recent years, DPI solutions were enhanced to include application-level traffic classification based on payload inspections and TLS metadata such as SNI. This technology allows analyzing network traffic encrypted with TLS/SSL protocols without decrypting it fully.

However, existing solutions are highly sophisticated and optimized, and thus, difficult to study due to their complexity. This work aims to develop a simplified DPI engine that will demonstrate the basic concepts behind DPI engines such as flow tracking, rules for classification, and multi-threaded processes.

IV. Proposed System

The proposed Packet Hunter System is a multi-threaded DPI (Deep Packet Inspection) Engine which analyzes the traffic on the network and classifies the packets using information obtained from the application level.

The architecture adopted uses a modular pipeline for packet analysis which includes acquiring packets, parsing the packets to identify protocol headers (ethernet, IP and TCP/UDP), analyzing the payload and identifying application level information by detecting the TLS handshake messages and extracting the Server Name Indication (SNI).

Flow Tracking is done to capture the connection-level information using a FiveTuple approach which includes IP address, ports and protocol of the packet. The flow contains information of its state like SNI, Application Type identified, and Classification state.

A Rule Based Engine is used to classify the traffic by applying the predefined rules which could be based on IP addresses, Domains or Applications. Based on the rules, the traffic can be allowed or flagged.

The proposed design is scalable and optimized for performance by adopting a multithreaded architecture, which uses the producer/consumer paradigm. The reader thread collects packets and distributes them to worker threads, using a load-balancing technique that involves the use of hashing on the FiveTuple.

The entire system has proven to be scalable and efficient, with flow-based tracking, packet analysis, and multithreading.

V. Methodology

The methodology adopted for this project involves adopting a sequential pipeline to handle the packets. This entails using different steps in analyzing and inspecting the data contained in the packets.

1) **Packet Collection:** Packets of network data are acquired from a predetermined source. These packets are processed individually.
2) **Packet Decomposition:** Packets are decomposed to obtain information about protocols such as Ethernet, IP, and TCP/UDP. Each header within the packet is obtained through offset-based parsing.
3) **Data Extraction:** Once all header information has been obtained, data from the payload section of the packet is extracted. This data is essential for deeper analysis.
4) **TLS Packet Identification and SNI Parsing:** The presence of TLS packets is detected, and Client Hello messages are identified within these packets. Binary parsing techniques are applied to extract Server Name Indication (SNI) from these packets.
5) **Flow Identification:** Every packet is associated with one unique flow via FiveTuple, i.e., the combination of source IP address, destination IP address, source port, destination port, and protocol.
6) **Flow Tracking:** Maintaining a flow table to keep track of the flow status such as extracted SNI, discovered application and the status of classification of the traffic is essential.
7) **Traffic Classification:** After discovering the SNI and flow details, the application type of the traffic is determined. Then, the classification engine is used to categorize traffic according to rules defined for domain/IP or application level.
8) **Decision:** Depending on the traffic classification performed before, traffic can be either accepted or marked.
9) **Multi-threading:** To increase efficiency, a multi-threaded architecture is implemented. Packets are shared between worker threads by the means of load balancing with respect to hashed FiveTuple values.

This methodology ensures efficient packet processing, accurate traffic classification, and improved scalability under high network load conditions.

VI. System Architecture

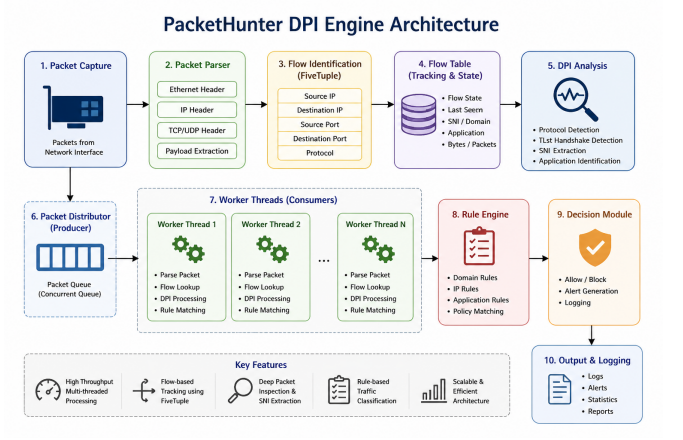

Fig. 1. Architecture of PacketHunter DPI Engine

VII. Implementation Details

The implementation of the *PacketHunter* system will be carried out in C++. The system will be optimized for efficient packet processing and designed with a modular approach. There will be a few components of the implementation each corresponding to different steps in the DPI process.

Packet Parsing: Packets are parsed using offset-based technique for extracting different header types including Ethernet, IP, and TCP/UDP headers. Pointer arithmetic is used for accessing different elements of packets efficiently.

Payload Processing: Once the packets are parsed and the headers are extracted, packets are processed by looking for TLS Handshake messages. In particular, Client Hello packets are found and the Server Name Indication (SNI) field is extracted using binary parsing.

Flow Tracking: Flow tracking is performed using an `unordered_map` which uses FiveTuple (Source IP address, Destination IP address, Source Port, Destination Port and Protocol) as the key to identify flows. Each flow will have different attributes associated with them including SNI, Application and Classification.

Rule Engine: The system makes use of rule-based classification implemented using `unordered_set` data structures. Traffic classification occurs on the basis of rules like blocking IPs, domains, and applications.

Multithreading: For better performance, the system makes use of multithreading via C++ `std::thread`. In order to achieve the task, a producer-consumer thread design is used in which a reader thread reads packets and distributes them amongst multiple threads.

Thread-Safe Queue: Communication between threads in the system takes place through the use of a thread-safe queue implemented via mutex and `condition_variable`.

Load Balancing: Packets are distributed amongst worker threads through a hash mechanism. FiveTuple is used to hash packets in order to make sure that all packets of the same flow get handled by the same worker thread.

Such an implementation makes the system modular and scalable while efficiently handling packet flows.

VIII. Results

Performance of the suggested system *PacketHunter* was estimated considering the capability to handle network packets fast in terms of load. The following characteristics were evaluated: throughput and processing efficiency.

Evaluation of the proposed system was performed by using test network traffic with several flows from various protocols. It can be noted that the multi-threaded implementation demonstrates better performance than the single-threaded one.

Thus, thanks to the utilization of the producer-consumer model and parallelism in the worker threads, the system managed to perform processing simultaneously, which provided the possibility to reach 2–3x faster throughput at the high load. At the same time, the flow-based approach allowed keeping track of the correct classification of all packets of the same connection.

Classification of traffic was conducted based on the rules according to the specified domain, IP, and application patterns with the help of the extracted SNI data. Thus, it is possible to say that the application level traffic identification is successfully provided for TLS connections.

All in all, the obtained results confirm that the DPI engine performs fast and efficiently.

IX. Advantages

The proposed system offers several advantages in terms of performance, scalability, and flexibility for network traffic analysis.

- **Effective Traffic Analysis:** Through deep packet inspection involving header analysis and payload analysis, the system can accurately classify application-level traffic.
- **Flow-Based Classification:** The FiveTuple-based flow management guarantees consistency in packet classification for packets within the same connection.
- **Scalability:** The multi-threading feature makes the system able to process packets in parallel and hence can handle high bandwidth traffic.
- **Enhanced Performance:** The producer/consumer paradigm and load balancing mechanism ensure efficient processing, resulting in performance gains of 2–3 times.
- **Modularity:** The design structure is modular, which allows the addition of more protocols or more sophisticated traffic inspection features.
- **Visibility at Application Level:** Extracting TLS Server Name Identification helps identify applications in encrypted traffic without decryption.

X. Limitations

Despite its effectiveness, the proposed system has certain limitations that can be addressed in future work.

- **Limited Protocol Support:** The present design is centered on handling TCP/UDP traffic and TLS encryption traffic. However, it only partially handles DNS traffic and HTTP traffic parsing.
- **No Real-Time Traffic Management:** While the system can analyze and classify traffic, it does not provide real-time management such as blocking or modifying any live traffic.
- **Dependency on SNI:** Identification of applications depends on Server Name Indication (SNI) extension of TLS encryption. It may not be available for newer TLS encryption standards.
- **Simulation Input:** The evaluation is done based on simulations and test inputs, rather than live input packets.
- **Memory Utilization:** Tracking the flow information using memory structure will require more memory in case of very high network traffic.
- **Limited Anomaly Detection:** The system depends on simple rule-based classification and does not use anomaly detection methods such as machine learning.

XI. Future Work

The proposed system can be further enhanced by incorporating additional features and improvements to increase its capability and real-world applicability.

- **Real-Time Packet Capture:** Compatibility with packet capture software like *libpcap* to facilitate real-time traffic analysis on live networks.
- **Enhanced Protocol Support:** Capability to analyze further protocols like DNS and HTTP for better traffic classification.
- **Traffic Analysis:** Use of machine learning algorithms for traffic analysis to detect anomalies and classify traffic.
- **Analysis of Encrypted Traffic:** Research into state-of-the-art methods like TLS fingerprinting (JA3) to analyze encrypted traffic.
- **Optimization:** Additional improvements in optimizing the program by adopting lock-free data structures and better load balancing techniques.
- **User Interface:** Design and implementation of a user interface for monitoring network traffic and system performance.
- **Dynamic Rule Configuration:** Configuring and updating rules in real time for increased flexibility and usability.

XII. Conclusion

In this paper, the design and implementation of *PacketHunter*, a multi-threaded Deep Packet Inspection (DPI) engine for analyzing network traffic, have been described. This work shows how one can analyze packet payload data, identify and extract application-specific information, including TLS SNI field value, and classify packets according to rules.

Flow tracking based on the FiveTuple technique has been utilized for managing connection-level state, guaranteeing consistent packet analysis within one flow. Moreover, the use of a multi-threaded design relying on the producer-consumer paradigm made possible efficient packet processing.

Based on experimental findings, it may be concluded that the system has better throughput performance under heavy load than its serial counterpart. Thus, the effectiveness of parallel computing and modular design has been demonstrated. This work illustrates some major concepts of modern network analysis tools and the implementation of a DPI engine.

References

[1] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, 1999.

[2] G. Varghese, *Network Algorithmics: An Interdisciplinary Approach to Designing Fast Networked Devices*. San Francisco, CA, USA: Morgan Kaufmann, 2005.

[3] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proc. 13th USENIX Conf. System Administration (LISA)*, 1999.

[4] N. McKeown et al., "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[5] J. Postel, "Transmission Control Protocol," RFC 793, 1981.